

ELI — Common Errors And Fixes

ELI v2.5.5

October 2026

Contents

ELI — Common Errors & Fixes	2
ELI stopped starting after a system upgrade: No module named ...	2
“ELI’s window cannot start: This system has no libGL.so.1”	2
undefined symbol: g_variant_builder_init_static / Failed to load module: ...libgvfsdbus.so	3
The Linux AppImage does nothing, or: version 'GLIBC_2.38' not found	3
From-source install on a clean system: stops early, or “complete” with no GUI	3
Voice is silent although a piper program is installed	3
GPU pack: libcudart.so.12: cannot open shared object file	4
GPU pack: no CUDA wheel found while offline-by-default	4
Windows: two ELI windows after Setup.exe	4
Setup wizard: nomic / Piper “won’t download”	4
Web server / phone “Home” page won’t connect	5
Microphone / voice input silent or “ELI can’t hear me” (desktop)	5
YouTube won’t play / “mpv is not installed” / no video window	5
Wayland: mouse clicks do nothing / ydotool errors	5
Clicking a link opens no browser page	6
Wrong / tiny model loads instead of your usual one	6
Server won’t start / “address already in use”	6
Phone can’t connect after a restart	6
Phone can’t connect at all (fresh pairing)	6
Phone microphone / voice doesn’t work	7
Model dropdown in the dashboard is empty	7
News shows old stories as “the latest”	7
Terminal shows escape-code junk around a URL (^[]8; ;...)	7
git push → “Could not resolve host github.com”	7
Vision / CLIP segfaults on the GPU	7
Arch / lean distros: “attempt to write a readonly database” (portable install)	7
Arch / lean distros: GUI won’t open — “libxcb-cursor0 is needed to load the Qt xcb platform plugin”	8
Running ELI on Arch — verified working steps	8
“ELI can’t see my Ollama models” (any OS)	8
ELI dies mid-session: double free or corruption / Aborted (core dumped)	9
The terminal floods with Audio source must be entered before listening	9
ELI reports facts about you, then calls them a hallucination when you push back	10
A reply ends abruptly on a dangling -	10
“ELI refuses a general question with ‘the net’s off, I can’t verify that’”	10
Natural voice / cloning: pip install "eli-v2.0[natural]" fails with “No matching distribution”	11
Natural voice / cloning: ImportError: cannot import name 'isin_mps_friendly'	11
Natural voice / cloning: hangs or EOFError on first use	11
Natural voice / cloning: ModuleNotFoundError: No module named 'torchaudio' (or torchcodec)	11
A community plugin won’t install	11

A plugin is installed but does nothing	12
A plugin asked for nothing and was denied anyway	12
An MCP server was added but has no tools	12
“ELI is offline” but a plugin still reached the internet	12
Training says my GPU can’t do it	13
“Install” opened a review dialog instead of just installing	13
A plugin is enabled but raises PermissionError	13
A registry or download was “refused for safety”	13

ELI — Common Errors & Fixes

Updated for v2.4.84 (September 2026). Primary path: GitHub Releases (ELI-Setup-*.exe, macOS dmg, Linux AppImage, portable ./ELI_Setup.sh). Wizard core stages hard-fail until nomic + voice + chat model are present. Windows: one desktop ELI + GUI singleton. **Home** tab ≠ Home Assistant.

A quick reference for the gremlins that actually come up when running ELI. Each one is symptom → cause → the exact fix. Add new ones as they surface.

ELI stopped starting after a system upgrade: No module named ...

Symptom (from-source install, any OS): ELI worked, the operating system or Python was upgraded, and now every start ends in ModuleNotFoundError: No module named 'requests' (or eli, PySide6, ...). On Windows the message is No Python at '...\python.exe'. **Cause:** ELI’s workspace (.venv) is tied to the Python it was built with. An upgrade that replaces that Python (Ubuntu 24.04 to 26.04 swaps 3.12 for 3.14; uninstalling or upgrading Python on Windows or macOS does the same) leaves the workspace starting as the new version, which cannot see a single installed package. **Fix:** start ELI the usual way. The launcher now says what happened and, if the old Python version is still installed anywhere (PATH, the py launcher, uv, pyenv, Homebrew), points the workspace back at it with nothing reinstalled. If it is not, run the installer once:

```
bash install.sh          # Linux / macOS
install.bat              # Windows
```

It rebuilds the workspace on the Python you have now. Models, memory and settings are not touched. To check or repair by hand: `python3 scripts/eli_env.py status / repair`. The release packages (AppImage, Setup.exe, dmg) bring their own Python and are not affected.

“ELI’s window cannot start: This system has no libGL.so.1”

Symptom: on a server, a container or a very minimal install the window does not open. Older versions printed Please install PySide6, which was not the reason. **Cause:** the graphics libraries (libGL, libEGL) belong to the GPU driver and the display server. ELI cannot carry its own copy without breaking hardware acceleration, so it uses the system’s, which every desktop has. **Fix:** run the command in the message, for example `sudo apt-get install -y libgl1 libegl1`. `install.sh` adds these itself when it can and the .deb depends on them. The server (ELI-Server) and terminal modes do not need them.

undefined symbol: g_variant_builder_init_static / Failed to load module: ...libgvfsdbus.so

Symptom: two warnings in the terminal at every start of the AppImage on a current distribution (Ubuntu 26.04, Fedora, Arch). Harmless, but file dialogs had no network places. **Cause:** the AppImage carried its own, older GLib, and GLib loads the system's GIO modules, which are built for the system's newer one. **Fix (this version):** the AppImage uses the system's GLib when the system has all of it and keeps its own for a system that has none. Nothing to do.

The Linux AppImage does nothing, or: version 'GLIBC_2.38' not found

Symptom: double-clicking the AppImage does nothing. From a terminal: Failed to load Python shared library ... libm.so.6: version 'GLIBC_2.38' not found. **Cause:** a Linux program only starts where the system library (glibc) is at least as new as the one it was built against. The AppImage is built on Ubuntu 24.04 and needs **glibc 2.39** or newer: Ubuntu 24.04+, Debian 13+, Fedora 40+, Linux Mint 22+, Pop!_OS 24.04+, RHEL / AlmaLinux / Rocky 10+, openSUSE Leap 16+ and Tumbleweed, Arch. Ubuntu 22.04 (2.35), Debian 12 (2.36), Mint 21 and RHEL 9 (2.34) are older. Check with `ldd --version`. **Fix:** on an older system install from source, which builds against the libraries the system has: download `ELI_v2-<v>-linux-portable.tar.gz` from the same release, unpack it and run `./ELI_Setup.sh`. Nothing can be added to the AppImage to make it start; upgrading the distribution also works. Current AppImages say this themselves, in a sentence and a dialog, instead of the loader's message.

From-source install on a clean system: stops early, or “complete” with no GUI

Symptoms (a clone or the source package, Linux): - The virtual environment was not created successfully because `ensurepip` is not available at the first step (stock Debian and Ubuntu). - `bash install.sh` ends without a message right after Installing ELI (editable), exit status 2 (any clone: there is no built wheel in dist/). - ERROR: Failed building wheel for PyAudio (Python.h: No such file or directory), the installer prints “installation complete”, and ELI then says Please install PySide6.

Cause: three separate faults on a system that has only Python. Debian and Ubuntu package the part of Python that makes environments separately. A lookup for a prebuilt wheel ended the script when there was none. And pip installs a set of requirements all or nothing: PyAudio has no ready-made Linux build and needs Python's headers and PortAudio's to compile, so without them none of ELI's dependencies were installed, while the final check passed because it imported ELI from the folder it was run in. **Fix (this version):** the installer first works out what it builds with and is missing, adds it when that takes no password and otherwise prints one command, for example

```
sudo apt-get install -y python3.12-venv python3.12-dev build-essential portaudio19-dev
```

then installs ELI itself whatever happens to its dependencies, falls back from the pinned set to version ranges and then to one requirement at a time (naming any that failed), and checks the result from outside the source folder. An install that ended with errors now says so and exits non-zero. On an older checkout: run the command above, `mkdir -p dist`, and run the installer again.

Voice is silent although a piper program is installed

Symptom: no speech, and the log shows `piper failed rc=1` or a traceback from a piper outside ELI's folder. **Cause:** more than one program is called piper. Some Linux distributions ship an unrelated one that configures gaming mice, and a piper installed for a Python that has since been removed still sits on PATH. ELI used to take the first one it found by name. **Fix (this version):** ELI uses the Piper

that was installed with it, and tries any other only if it starts and is the speech engine. Nothing to do. ELI_PIPER_BINARY=/path/to/piper still forces a particular one.

GPU pack: libcudart.so.12: cannot open shared object file

Symptom: AppImage or portable dies (or eli CLI dies) with Failed to load ... libllama.so: libcudart.so.12: ... No such file or directory. Disk shows ~/.local/share/ELI_v2/runtime/gpu ~2.7 GB with libggml-cuda.so but **no** libcudart.so.12 / libcublas.so.12. Iris Xe / AMD may still log Vulkan backend forced while installing a CUDA wheel. **Cause (≤2.4.25):** the --vulkan / Intel / AMD picker preferred cuda- gpu-packs assets, labelled them vulkan, and skipped NVIDIA runtime vendoring. **Fix (v2.4.26+):** upgrade; remove the broken pack and relaunch so the correct backend is downloaded:

```
rm -rf ~/.local/share/ELI_v2/runtime/gpu
# Launch the new AppImage (or: AppImage --integrate then eli)
```

NVIDIA machines get a CUDA pack **with** vendored cudart/cublas (or install fails closed). Intel/AMD/--vulkan get a real vulkan- wheel only.

Also check which eli — an old portable wrapper can still be first on PATH even after installing a newer AppImage. Run ./ELI_v2-* -x86_64.AppImage --integrate so ~/.local/bin/eli points at the AppImage you just installed.

GPU pack: no CUDA wheel found while offline-by-default

Symptom: Startup dialog: *GPU pack install failed: no CUDA wheel found...* Logs show index cu124 unavailable (network disabled (offline mode): blocked connection to ('abetlen.github.io', 443)) on a machine that *does* have NVIDIA. **Cause (≤2.4.25):** GPU pack install used raw urlopen without a scoped allow_network() window, so NetGuard blocked the wheel index. **Fix (v2.4.26+):** install opens allow_network("gpu-pack install") and falls back to CI cuda- packs on the gpu-packs GitHub release if abetlen is empty. On portable, if ggml_cuda_init already lists your GPU, inference can still use the venv CUDA build — the dialog failure is about the optional GPU pack, not that CUDA is impossible.

Windows: two ELI windows after Setup.exe

Symptom: Finish the installer and two GUI windows appear (or desktop has ELI + ELI Server). **Cause:** Desktop used to pin both ELI.exe and ELI-Server.exe; post-install could race. **Fix (v2.4.23+):** Desktop shortcut is **ELI only**; Server stays in Start Menu. Frozen GUI takes a singleton lock — a second launch shows “ELI is already running” and exits. Close the extra window; upgrade to 2.4.23+ (current **2.4.84**).

Setup wizard: nomic / Piper “won’t download”

Symptom: Chat model downloads, but embedder/voice stay missing. **Cause:** Older wizard called download_aux(required_only=False) and ignored failures; offline-by-default blocked FirstBoot prefetch. **Fix (v2.4.23+):** Wizard uses python -m eli.core.model_download --aux with a real exit code. Click **Fetch embedder + voice now** (scoped network) or run:

```
.venv/bin/python -m eli.core.model_download --aux
.venv/bin/python -m eli.runtime.voice_assets
```

Web server / phone “Home” page won’t connect

Symptom: Settings → Web Server starts, phone can’t open the URL / QR is <this-computer-ip>. **Cause:** LAN IP detection lacked a Windows path; Apply-port left a stale ELI_API_PORT; HTTPS sidecar couldn’t stop; Windows firewall tip omitted 8443. **Fix (v2.4.23+):** Shared resolve_lan_ip() (UDP + PowerShell); Stop clears port env; HTTPS handle is retained; firewall hints include HTTPS. Use **phone / Wi-Fi** mode, allow the firewall command, open the **token** URL (Home tab = devices/MQTT, not Home Assistant).

Microphone / voice input silent or “ELI can’t hear me” (desktop)

Symptom: ELI starts, [AUDIO] Listening... appears, but nothing is transcribed — or you had to set ELI_MIC_DEVICE_INDEX manually on v2.3.30. **Cause (v2.3.30 regression):** on Linux/PipeWire, PortAudio’s pulse wrapper often passed a short probe yet stayed silent during real STT; on all platforms a webcam or virtual “Stereo Mix” default could win over a connected USB/Bluetooth headset. **Fix (v2.3.92+):** upgrade to **v2.3.92** — cross-platform ranked auto-resolve probes USB, headset/Chat, and Bluetooth endpoints before built-in/webcam/virtual routes, at each device’s native sample rate. No env vars required on Linux, macOS, or Windows. **Diagnostics:**

```
python -m eli.tools.mic_diag          # lists devices + measures speech vs ambient
```

Manual override (only if auto-resolve picks the wrong endpoint):

```
python -m eli.tools.mic_diag          # note the [N] of your working device
export ELI_MIC_DEVICE_INDEX=N         # use YOUR index from mic_diag – not a fixed number
# Linux PipeWire/Pulse pin (optional):
export ELI_MIC_PULSE_SOURCE="$(pactl get-default-source)"
```

On Windows gaming headsets, set the **Chat** mic (not Game) as the default recording device in Sound settings. On macOS, pick your AirPods or headset under System Settings → Sound → Input.

YouTube won’t play / “mpv is not installed” / no video window

Symptom: “play X on YouTube” fails, opens browser only, or plays audio with no video. **Cause:** mpv and/or yt-dlp missing. yt-dlp is a Python package (installed into ELI’s bundled interpreter); mpv is an OS package. **Fix:** Say **yes** when ELI offers to install after a failed attempt, or run manually:

```
# Linux
sudo apt install mpv      # or dnf/pacman equivalent
# yt-dlp is installed by install.sh into ELI's Python – or: python -m pip install yt-dlp
# macOS
brew install mpv
# Windows
winget install mpv.MPV
```

Note: Plain “play X on youtube” uses **visible mpv**. “play X on youtube.com” opens the browser.

Wayland: mouse clicks do nothing / ydotool errors

Symptom: MOUSE_CONTROL or screen locate clicks fail on GNOME/KDE Wayland; error mentions ydotool. **Cause:** xdotool only reaches XWayland windows. Native Wayland needs ydotool + the ydotool daemon with /dev/uinput access. **Fix:**

```
sudo apt install ydotool      # or your distro equivalent
ydotoold &                   # run daemon (may need uinput group)
```

Or use AT-SPI/OCR click paths for labelled controls. See docs/CROSS_PLATFORM.md.

Clicking a link opens no browser page

Symptom: the server prints the URLs fine and copy-paste works, but Ctrl/right-click → “Open Link” opens nothing. **Cause:** broken **snap Firefox** — a mesa-2404 auto-refresh leaves Firefox’s GPU content-mount stale (/snap/firefox/.../gpu-2404-provider-wrapper goes missing), so *launching* a fresh Firefox fails silently while an already-open window still works. **Fix:**

```
killall -9 firefox
sudo snap disconnect firefox:gpu-2404
sudo snap connect firefox:gpu-2404 mesa-2404:gpu-2404
# if it's still stuck:
sudo snap refresh firefox
sudo snap refresh mesa-2404
```

Reopen Firefox once, then relaunch the server — clicks (and the auto-open) work again. (*Nothing wrong with ELI — the URLs and server are fine; it’s the snap.*)

Wrong / tiny model loads instead of your usual one

Symptom: ELI boots on a small model (e.g. tinyllama-1B) even though settings point at your 35B A3B. **Cause:** a stray ELI_GGUF_MODEL_PATH / ELI_MODEL_PATH env var in your shell **overrides config** (it wins over everything). **Fix:**

```
echo $ELI_GGUF_MODEL_PATH          # if this prints a model path, that's the culprit
unset ELI_GGUF_MODEL_PATH ELI_MODEL_PATH
```

Or just switch live from the dashboard: **Settings** → **Model**.

Server won’t start / “address already in use”

Cause: a previous instance is still bound to the port. **Fix:**

```
killall -f "api/server.py"          # or find it: ss -ltnp | grep 8081
```

Then relaunch. ELI’s web port is **8081**, HTTPS **8443**.

Phone can’t connect after a restart

Symptom: a paired phone gets 401 / “can’t access the server.” **Cause:** it’s carrying an old token (server restarted, or you rotated the token). **Fix:** the token now **persists** across restarts — just re-open the current link/QR from the **Connect** tab. To deliberately kick a lost phone off, hit **rotate** and re-pair.

Phone can’t connect at all (fresh pairing)

Cause: the firewall is blocking the port. **Fix:** run the exact `sudo ufw allow ...` lines the server prints on startup, and make sure the phone is on the **same Wi-Fi**.

Phone microphone / voice doesn't work

Cause: browsers block the mic (getUserMedia) on a plain http://LAN-IP page. **Fix:** start with --https and open the https://...:8443/ link on the phone; accept the one-time self-signed “not private” warning.

Model dropdown in the dashboard is empty

Cause: the list endpoint was being shadowed by the OpenAI-compatible /v1/models route. **Fix:** already fixed — the list lives at /v1/models/installed. If a fork regresses it, make sure the dashboard fetches that path, not /v1/models.

News shows old stories as “the latest”

Cause: the interest-matched half wasn't recency-gated. **Fix:** already fixed (a freshness gate drops stale niche matches). If it recurs, say refresh the news.

Terminal shows escape-code junk around a URL (^[]8;;...)

Cause: you're looking at a **log file** or cat -v output — those *always* render raw escape codes. A live terminal renders the link normally. Not a bug.

git push → “Could not resolve host github.com”

Cause: a transient DNS/network blip (ELI is offline-by-default, but git is separate). **Fix:** just retry the push.

Vision / CLIP segfaults on the GPU

Cause: the CLIP/vision path can segfault on some GPUs. **Fix:** CLIP runs on **CPU** by design; keep it there. Main-model + vision hot-swap handle VRAM automatically — don't force CLIP onto the GPU.

Arch / lean distros: “attempt to write a readonly database” (portable install)

Symptom: on the portable Linux build, first-run DB init (Step 5) fails for the user.* stores with sqlite3.OperationalError: attempt to write a readonly database, while system_index / coding_memory / agent succeed in the same folder; the GUI then can't open the memory DB and exits. **Cause:** the folder you extracted ELI into lives on a filesystem whose locking SQLite's **WAL journal** needs isn't supported — **NTFS, exFAT/FAT, or a network mount** (very common for a ~/Downloads on a dual-boot data drive). WAL returns SQLITE_READONLY there; the rollback-journal stores beside it work fine, which is the tell-tale split. (This is also why the AppImage — which stores data under ~/.local/share/ELI_v2 on your main partition — worked while the portable build in ~/Downloads didn't.) **Fix:** fixed in **v2.1.19** — ELI detects a WAL-hostile filesystem and falls back to a DELETE (rollback) journal automatically, so it runs wherever you put it. On an older build, extract ELI onto an **ext4/btrfs** path (e.g. under your home partition) instead. Confirm your mount with: findmnt -T . -o TARGET,FSTYPE.

Arch / lean distros: GUI won't open — “libxcb-cursor0 is needed to load the Qt xcb platform plugin”

Symptom: the AppImage's engine loads (you see the ELI v2.0 banner and the module log) but the window never appears; it aborts with the xcb-cursor0 ... xcb platform plugin message. **Cause:** Qt 6.5+'s xcb platform plugin (libqxcb.so) links the **whole xcb-util family** — libxcb-cursor, libxcb-icccm, libxcb-image, libxcb-keysyms, libxcb-render-util, libxcb-util — which a minimal desktop doesn't install. **Qt's message is misleading:** it always blames libxcb-cursor0 even when the real missing library is a different member (on a test Arch box the actual culprit was libxcb-icccm.so.4). **Fix:** fixed in **v2.1.21** — the AppImage now bundles the full xcb-util family (in PySide6/Qt/lib), so it launches out of the box on bare Arch/Debian with no extra packages. On an older build, install them yourself:

```
# Arch
```

```
sudo pacman -S xcb-util-cursor xcb-util-wm xcb-util-image xcb-util-keysyms xcb-util-renderutil xcb-util
```

```
# Debian/Ubuntu
```

```
sudo apt install libxcb-cursor0 libxcb-icccm4 libxcb-image0 libxcb-keysyms1 libxcb-render-util0 libxcb-util
```

Diagnose exactly which library is missing (Qt's own trace names it):

```
QT_DEBUG_PLUGINS=1 ./ELI_v2-*-*x86_64.AppImage 2>&1 | grep -iE 'cannot|not found'
```

Running ELI on Arch — verified working steps

The **AppImage** is the easiest path: it bundles its own **Python 3.11**, so Arch's system Python 3.14 (which has no llama-cpp-python wheel) is irrelevant, and as of **v2.1.21** it bundles every Qt xcb library too. Download and run:

```
U=https://github.com/ShadowESC95/ELI_v2.0/releases/download/v2.4.84
```

```
wget "$U/ELI_v2-2.4.84-x86_64.AppImage"
```

```
chmod +x ELI_v2-2.4.84-x86_64.AppImage
```

```
./ELI_v2-2.4.84-x86_64.AppImage
```

Run it directly (as above) so it FUSE-mounts in place — `--appimage-extract-and-run` unpacks ~4 GB into /tmp, which fails with `libz.so.1: file too short` when /tmp is a small tmpfs. If you launch it over **SSH** into a running desktop session, prefix the command with `export DISPLAY=:0`. (Paste long URLs as a **single line** — a wrapped URL 404s and the tail runs as a stray command.)

Verified end-to-end on a clean Arch VM (XFCE): engine loads, database initialises on a WAL-hostile disk via the DELETE fallback, and the GUI opens with **every** xcb-util package uninstalled.

“ELI can't see my Ollama models” (any OS)

Symptom: Backend is set to Ollama but the model list is empty, or ELI reports Ollama unreachable ([Errno 111] Connection refused) while Ollama is installed.

Fix (v2.1.30): ELI now starts Ollama for you. The #1 cause is simply that the local Ollama server isn't running. If the ollama program is installed but stopped, ELI now runs `ollama serve` automatically when you pick/refresh Ollama, waits for it, and loads your models — no terminal needed. If it still can't start (Ollama not installed, or a remote host), you get per-OS guidance. Disable the auto-start with `ELI_OLLAMA_AUTOSTART=0`. Full walkthrough for non-technical users: [blueprints/ollama_startup_guide.md](#).

Fix (v2.1.31): the toolbar dropdown itself was broken. Separately from the server ever starting, the toolbar's Ollama dropdown could sit permanently empty with the status dot stuck on “Checking Ollama...” *even when Ollama was running and serving models perfectly*. The widget fetched the list on a background thread and handed the result back with `QTimer.singleShot(0, ...)` — but a Qt timer started

from a plain threading.Thread has no Qt event loop to run on, so the callback was silently dropped and the combo was never filled. It now hands results back over a Qt **signal**, which queues to the GUI thread correctly. The model-pull progress dialog had the same defect (progress never advanced, dialog never closed) and was fixed the same way. Regression-tested in tests/test_ollama_selector_threading.py, including a guard that fails if a future refactor reintroduces a timer on that path. Note this was invisible to the existing GUI tests because they only asserted the widget *constructs* — the new tests assert models actually land in the dropdown.

Earlier address-handling fixes (still in place) — you no longer need to do anything special about how the host is written:

- **OLLAMA_HOST set the way Ollama documents it** (OLLAMA_HOST=127.0.0.1:11434, with no http://) used to break ELI with unknown url type. Scheme-less hosts, bare IPs and missing ports are now all understood, everywhere — client, startup picker, wizard, Settings.
- **localhost resolving to IPv6 first** (common on Windows) while Ollama listens on IPv4 — ELI now automatically retries 127.0.0.1, so this no longer looks like “Ollama is down”.
- **Ollama on another machine** was blocked by ELI’s offline-by-default guard with no explanation. A host you configure is now treated as a deliberate local service, like a LAN MQTT broker, so it is allowed and the Net toggle can stay OFF.

If it is still not found:

```
ollama list           # is it running, and does it have models?
ollama pull llama3.2  # pull one if the list is empty
```

Per-OS start: **Windows/macOS** — Ollama starts itself (tray / menu bar); launch “Ollama” if it isn’t running. **Linux** — systemctl --user start ollama, or ollama serve.

For Ollama on a **different machine**, on that machine:

```
OLLAMA_HOST=0.0.0.0:11434 ollama serve # listen beyond localhost
```

and allow port 11434 through its firewall. Then in ELI set **Settings → Model → Ollama → Host** to that machine (e.g. 192.168.1.20 — the port is filled in for you).

ELI dies mid-session: double free or corruption / Aborted (core dumped)

Symptom: the GUI is running fine, then the terminal prints a run of Warning: maximum number 350 of (N_VOICES_LIST = 350 - 1) reached, followed by double free or corruption (!prev) and Aborted (core dumped). The whole app is gone.

Cause: not ELI. espeak-ng has a hardcoded N_VOICES_LIST of 350. On a machine with more installed voice/variant combinations than that, espeak_ListVoices walks off the end of its array and corrupts the heap. ELI reached it through pyttsx3.init() while listing system voices. Because that is a **native** abort(), the Python try/except around it caught nothing — the crash took the entire GUI with it.

Fix (v2.1.32): system-voice enumeration now runs in a **subprocess**. If espeak crashes, it kills a throw-away child; ELI logs the non-zero exit and continues with no system voices listed. Nothing to configure.

The terminal floods with Audio source must be entered before listening

Symptom: thousands of identical [AUDIO] Listen error: Audio source must be entered before listening... lines, a pinned CPU core, and a log that grows without limit.

Cause: listen_once() and the mic-calibration helper open with self.microphone on the **same** Microphone object the background listen loop is holding. speech_recognition’s __exit__ sets stream = None, so when either finishes it closed the stream out from under the running loop — which then failed on every iteration, forever, at full speed.

Fix (v2.1.32): the loop detects the closed stream and **re-enters** it instead of spinning, and that error is rate-limited (once per 5s with a count) so no fault can bury the log again.

ELI reports facts about you, then calls them a hallucination when you push back

Symptom: ELI gives a grounded profile answer, you say “wrong”, and it retracts everything — “that was likely a hallucination”, “I don’t have access to your personal files”, “I only see the text in this window”.

Cause: those statements are **false**. The facts came out of `user.sqlite3`. This is the base model’s generic cloud-assistant reflex overriding what ELI actually is. There was a rule against *inventing* mechanisms, but none against *denying real ones* — the mirror failure.

Fix (v2.1.32): a NO FALSE SELF-DENIAL rule. Push-back is not proof ELI was wrong: it must correct the **specific** disputed field and say where the stored value came from, never retract a whole grounded answer as imaginary. The phrases also now trip the evidence validator. A companion rule stops “what do you know about yourself” being answered with *your* profile.

A reply ends abruptly on a dangling -

Symptom: a list-shaped answer stops mid-structure with an empty bullet on the last line.

Cause: the generator hit its token cap just as it opened the next bullet. The truncation detector missed it (a - is neither alphanumeric nor a sentence terminator, so it read as a clean ending), and the re-generation repair is deliberately **non-quick only** — a second inference would defeat quick mode’s latency.

Fix (v2.1.32): the detector now recognises an orphan list marker, and the output governor trims it in **every** mode, so a capped answer ends on its last complete line.

“ELI refuses a general question with ‘the net’s off, I can’t verify that’ ”

Symptom: an everyday advice/how-to question — “*what is the best way to eat breakfast*” — is met with “*I can’t verify that right now — the net’s off ... turn the Net toggle on*”, and with the Net on it returns an irrelevant web snippet (a user saw “*Best Buy doesn’t offer breakfast recommendations*”).

Cause: the fact classifier matched the “**what ... is**” shape and treated the advice question as a checkable external fact (oddly, the apostrophe form “*what’s the best way...*” slipped through and answered normally). So offline it hit the honest can’t-verify hedge, and online it was web-searched — surfacing whatever the search returned.

Fix: two layers, no action needed on your part. - **v2.1.25** — advice / how-to / recommendation questions (“best way to...”, “how do I...”, “how to...”, “what should I...”, “tips for...”, “should I...”) are no longer classed as external facts. They’re answered directly from the model’s own knowledge, offline, like any other chat. Genuine facts (“what is the capital of Japan”, “how old is ...”, “who won ...”) still verify as before. - **v2.1.26** — a *relevance gate* on web results: when a genuine fact is searched and the search returns topically-unrelated pages (the “Best Buy” case), ELI no longer synthesises them into a confident non-answer. It says it searched but couldn’t find anything that answers the question, and asks you to rephrase or add a detail — an honest miss instead of a made-up one. Tune with `ELI_WEB_RELEVANCE_FLOOR` (default 0.34; higher = stricter).

Natural voice / cloning: `pip install "eli-v2.0[natural]"` fails with “No matching distribution”

Symptom: following an in-app message or older doc literally (`pip install "eli-v2.0[natural]"`) fails — `pip` tries to fetch `eli-v2.0` from PyPI, where this project isn’t published. **Cause:** that string only resolves once `eli-v2.0` is already installed under that name from a real index; a source checkout needs the local-path form instead. **Fix:** from the ELI project root: `pip install -e "[natural]"` (or `[clone]/[voice]` — all three are aliases for the same Coqui XTTS-v2 extra). Fixed at the source in v2.1.29 — every in-app message now shows the correct form.

Natural voice / cloning: `ImportError: cannot import name 'isin_mps_friendly'`

Symptom: after installing the `[natural]/[clone]` extra, the first clone/natural-voice attempt raises `ImportError: cannot import name 'isin_mps_friendly' from 'transformers.pytorch_utils'`. **Cause:** `coqui-tts` (latest release, 0.27.5) still calls a `transformers` helper that existed only as an old Apple-MPS `torch.isin()` fallback and was removed in `transformers>=5` — the exact version ELI’s own lock file pins for the rest of the stack, so downgrading `transformers` isn’t an option. **Fix:** fixed in v2.1.29 — `eli/perception/tts_xtts.py` restores the function with a lazy compat shim (plain `torch.isin()`, identical behaviour off MPS) before TTS is imported. No action needed on an up-to-date checkout.

Natural voice / cloning: hangs or `E0FError` on first use

Symptom: the very first clone or natural-voice synthesis after installing the extra either hangs indefinitely or raises `E0FError`: `E0F` when reading a line, and the GUI/voice command silently falls back to a normal voice instead. **Cause:** `coqui-tts` gates the first XTTS-v2 model download behind an interactive y/n terminal prompt (agree to the non-commercial Coqui Public Model License, or confirm a paid licence). ELI’s GUI has no terminal for that prompt to read from. **Fix:** fixed in v2.1.29 — `tts_xtts._get_model()` sets `COQUI_T0S_AGREED=1` before loading, which is always the correct branch here (a local, personal voice, never redistributed by ELI). No action needed on an up-to-date checkout; using this feature at all implies agreeing to the non-commercial CPML (a commercial deployment needs a separate paid licence directly from Coqui).

Natural voice / cloning: `ModuleNotFoundError: No module named 'torchaudio' (or torchcodec)`

Symptom: `xtts_available()` / a clone attempt fails with a missing `torchaudio` or `torchcodec` import, even though `coqui-tts` installed successfully. **Cause:** `coqui-tts` doesn’t declare `torch/torchaudio/torchcodec` as install dependencies (same reasoning as ELI’s own training extra — a GPU build must match your CUDA, so it can’t be hard-pinned), but genuinely needs all three at import time. **Fix:** fixed in v2.1.29 — the `clone/natural/voice` extras now pull in `torch`, `torchaudio`, and `torchcodec` themselves, so a plain `pip install -e "[natural]"` on a fresh machine resolves everything together. If you already had a bleeding-edge `torch` installed *ahead* of the latest published `torchaudio/torchcodec` (e.g. a nightly build), install the matching version manually from <https://download.pytorch.org/whl/<your-cuda-tag>> — same pattern as the main `torch` line in `requirements.txt`.

*House rule: when a new error bites, add it here — symptom, cause, and the **exact** commands that fixed it. Future-you will thank present-you.*

A community plugin won’t install

ELI refuses at the first stage that fails, and the dialog names it:

Stage	Meaning	What to do
listing	the registry entry is malformed	the publisher must fix their manifest
payment	paid plugin, no licence key stored	buy from the publisher, then enter the key
download	source URL unreachable	check the URL, and that networking is on
integrity	checksum or signature mismatch	do not retry — the file does not match what the listing described
code	uses a capability it never declared	the publisher must declare it; do not force this
malware	a scanner found malicious indicators	do not install
dependencies	wants PyPI packages	approve separately with the pip-inclusive button
consent	nothing was available to ask you	install from the GUI, not headless

An integrity or malware refusal is not a bug to work around. Both mean the file on the server is not the file the listing promised, or is actively hostile.

A plugin is installed but does nothing

Installed plugins arrive **switched off with no permissions granted** — that is deliberate. Enable it in Settings ► Marketplace ► Installed. It will then ask, one capability at a time, the first time it needs each one.

If you chose **Never allow** for a capability, the plugin is never asked again and will keep failing silently at that step. Revoke the decision in Settings ► Marketplace ► Permissions to be asked afresh.

A plugin asked for nothing and was denied anyway

Permission requests **fail closed** when there is nothing to ask: a headless run, the API server, or a scheduled overnight task. That is intended — a plugin must not gain a capability by running at 3am. Run it once from the desktop GUI and choose **Always allow** if you want it to work unattended.

An MCP server was added but has no tools

Run the doctor (Settings ► Marketplace, or `eli.plugins.mcp.doctor()`). It reports each configured server with the exact fault:

- **runtime missing** — `npm/uvx` is not installed or not on `PATH`. The message names the install command for your platform.
- **handshake timeout** — the process started but never answered MCP initialize. Usually the wrong command, a missing argument, or a required environment variable.

Note that installing already runs a real handshake, so a server that was accepted did answer at least once. There is exactly one config file — `mcp.config_path()` — and ELI reads no other; a previous config from a different host will not be picked up.

“ELI is offline” but a plugin still reached the internet

Expected, and important to understand. `netguard` patches Python’s socket layer **inside ELI’s process**. An MCP server, a pip install, or any other child process has its own network stack and is not affected.

ELI's offline switch does not stop them and ELI cannot see what they send. Only install MCP servers you trust with that. See `security.md` §17.

Training says my GPU can't do it

Read the sentence — it names the shortfall and the fix, e.g. *“6.37 GiB free but this run needs ~9.04 GiB.”* Options, in order of effort: close what is using VRAM, lower the sequence length in Advanced, install bitsandbytes for 4-bit training, or pick a smaller base model. CPU training works but takes hours to days, which is why it is never selected silently.

“Install” opened a review dialog instead of just installing

That is the design. The one-click path proceeds only when there is nothing to decide. It stops, and names the reason, when: the scan is not clean, the listing has no checksum, the plugin wants permissions, it installs PyPI packages, it costs money, or its download URL is plain http. Missing optional scanners and an unsigned plugin do **not** stop it — they are reported afterwards.

A plugin is enabled but raises `PermissionError`

Runtime enforcement is doing its job. `eli/plugins/sandbox.py` blocks an operation whose capability the plugin's manifest never declared — including one made by importing `socket` or `subprocess` directly rather than through ELI's API. The message names the plugin, the operation and the missing capability.

If the plugin genuinely needs it, the publisher must declare it in the manifest; there is deliberately no way for you to grant a capability the plugin never asked for. `ELI_PLUGIN_SANDBOX=0` disables enforcement entirely and should only be used to confirm that this is what is happening.

A registry or download was “refused for safety”

`netguard.safe_fetch` refuses URLs that are not http(s), and hosts that resolve to loopback, private, link-local or reserved addresses — including after a redirect. This stops a listing you do not control from pointing ELI at your own machine or LAN.

If it is your own registry on your own network, add it as a source: `add_registry` detects a local address, records the exception explicitly, and warns you. A public registry that then redirects somewhere private is still refused.